

DATA 542 – Analysis of Autonomous AI Agents in Software Development: Success, Efficiency, and Evolution

TAO WANG, University of British Columbia, Canada

This project looks at how five autonomous AI coding agents behave when creating pull requests on GitHub. We used a pretty large dataset (around 930k PRs). Basically, we wanted to figure it out: how often PRs got merged, how fast they were merged, and also if any agent improved month by month.

OpenAI_Codex shows the strongest and most steady performance, while Devin and Copilot start lower but show some signs of getting better in later months. We also notice that longer PR descriptions help a few agents a little bit, though the effect is not very strong and not the same for all of them.

These patterns suggest that the agents do not follow the same workflow style, and some of them keep changing or stabilizing as time goes on.

1 Introduction

With the continuous updates and iterations of AI coding tools, many development teams now rely on these AI coding tools for rapid prototyping and daily programming tasks. But, it's honestly not very clear if different AI agents write code that humans find equally easy to review. We wanted to take a closer look at how these agents behave in real GitHub workflows.

This project focuses on three questions.

- **RQ1:** How do the five autonomous coding agents differ in pull-request success rate and efficiency? It can tell us how often their pull requests (PRs) get merged and how quickly.
- **RQ2:** Does the length of the pull-request description affect the chance of being merged, and does this effect vary across agents? It can tell us whether writing longer descriptions actually helps an agent get merged.
- **RQ3:** Did any of the agents improve over time (Dec 2024 – Jul 2025)? It can tell us whether certain agents become more reliable across time, possibly due to model updates.

2 Methodology

In this section, I describe how we approached each research question and the reasoning behind our choices.

2.1 RQ1: How do the five autonomous coding agents differ in pull-request success rate and efficiency?

To compare the agents, we first focused on closed PRs and created two summary measures for each one:

- the merge rate (the proportion of closed PRs that were merged),
- the median time-to-merge in hours (computed only for merged PRs).

We used a chi-square test to check whether merge rates differed across agents.

At first, I planned to run a standard ANOVA, but after checking the assumptions, the dataset is obviously not normally distributed. So I switched to the Kruskal-Wallis test since it fits this dataset better.

To illustrate the findings, we plotted a bar chart of merge rates and a set of boxplots showing the distribution of time-to-merge for each agent.

2.2 RQ2: Does the length of the pull-request description affect the chance of being merged, and does this effect vary across agents?

To study whether longer descriptions help PRs get merged, and whether this varies by agent, we fitted a logistic regression model:

$$\text{merged} \sim \log(\text{body_length}) * \text{agent}$$

The interaction terms allow each agent to respond differently to increases in description length.

Since regression coefficients can be hard to interpret on their own, we also divided PRs into four description-length quartiles (from Q1 shortest to Q4 longest) and compared merge rates across agents within each quartile.

2.3 RQ3: Did any of the agents improve over time (Dec 2024 – Jul 2025)?

For the temporal analysis, we extracted the year-month from each PR's creation timestamp and computed, for every agent and month:

- the number of PRs submitted,
- the merge rate for that month.

We then plotted the month-by-month merge-rate curves and calculated Spearman correlations between the month index and each agent's merge rate to look for monotonic trends. Some early months contained only a small number of PRs for certain agents, so we interpreted those fluctuations with caution.

3 Results

3.1 RQ1: How do the five autonomous coding agents differ in pull-request success rate and efficiency?

Comparing these agents, differences are really big. Figure 1(a) shows us that OpenAI_Codex had the highest merge rate at roughly 93.8%, with Cursor (about 89.8%) and Claude_Code (about 83.3%) following behind. Copilot (about 74.09)

In terms of efficiency, Figure 1(b) shows us that OpenAI_Codex and Cursor performed best. Their median time-to-merge values were extremely short (on the order of a few minutes), whereas Copilot's median time-to-merge was noticeably longer (about 0.62 hours). Because the time distributions had many extreme values, we used the Kruskal-Wallis test, which indicated clear differences across agents ($p < 0.05$).

The results show that there was kind of a pattern: the agents that merged more often also tended to merge faster.

3.2 RQ2: Does the length of the pull-request description affect the chance of being merged, and does this effect vary across agents?

The regression results show that description length had a small but noticeable relationship with merge probability. The interaction terms were helpful because they revealed that agents did not react to longer descriptions in the same way.

For Copilot, the interaction term was positive and statistically significant, meaning that writing a longer description gave its PRs a slightly better chance of being merged. Cursor, however, showed the opposite pattern. Its negative interaction term suggests that very long descriptions did not help and may even have reduced its merge chance a little. For Devin and OpenAI_Codex, the interaction effects were small and statistically weak, implying that extra text did not change their outcomes in any meaningful way.

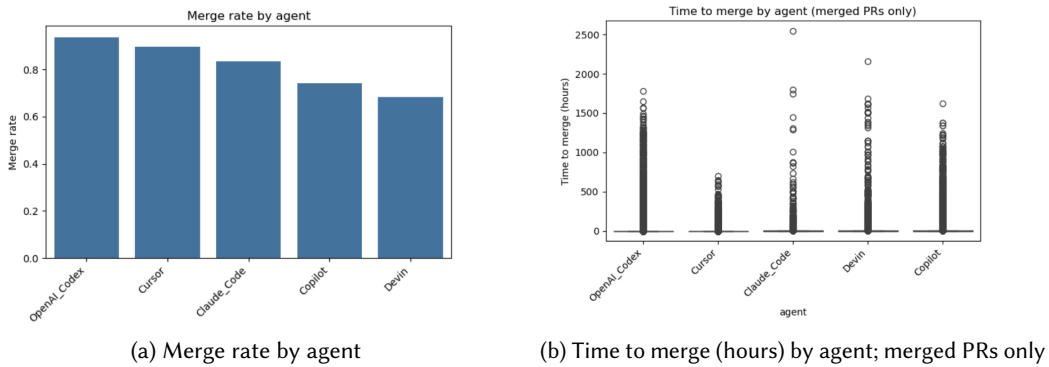


Fig. 1. Pull-request success rate and efficiency patterns across agents.

Since logistic regression coefficients can be difficult to interpret on their own, we also grouped PRs into four description-length quartiles and compared merge rates across agents. This visualization (Figure 2) made the differences clearer. Cursor and Claude_Code showed slightly higher merge rates in the longer-description groups (Q3–Q4), while Copilot and Devin displayed more irregular patterns. Some of this variation is likely due to uneven sample sizes across quartiles, especially for agents with fewer PRs.

The results suggest that description length plays only a minor role, and the effect depends on which agent we choose to use.

3.3 RQ3: Did any of the agents improve over time (Dec 2024 – Jul 2025)?

When we look at month-by-month performance, a few agents showed clear upward trends. OpenAI_Codex had the strongest pattern, with a nearly perfect positive Spearman correlation ($\rho = 1.00$, $p < 0.001$). Devin also improved over time ($\rho \approx 0.88$, $p \approx 0.003$), and Copilot displayed a moderate positive trend ($\rho \approx 0.90$, $p \approx 0.04$).

In contrast, Cursor and Claude_Code did not show meaningful improvement. Their merge rates fluctuated from month to month, and several of the early months contained very few PRs (in some cases, fewer than ten). With such small sample sizes, even a couple of merges can shift the monthly rate noticeably, so we interpreted these fluctuations cautiously.

Overall, the results suggest that Codex, Devin, and Copilot became more reliable across the observed period, while Cursor and Claude_Code stayed relatively stable.

4 Discussion

When we look across the results from all three questions, a few themes became clearer.

I think OpenAI_Codex worked the best because it had high acceptance rates. It had high acceptance rates, quick review times, and steady performance each month. Devin, on the other hand, started out much weaker but showed one of the clearest upward trends over time, which makes its trajectory more interesting than its raw merge rate alone would suggest. Copilot’s results were less steady. Its merge rate dropped early but later recovered, which was unexpected. Actually, I was surprised by this since I expected all agents to be similar at first.

The results from RQ2 added more detail to the picture. At beginning, I first thought longer descriptions would always help, but data showed me wrong in some cases. Overall, the effect was small, and each agent reacted differently to longer descriptions. For example, Cursor seemed to benefit from longer explanations, while Codex did well no matter how detailed the description

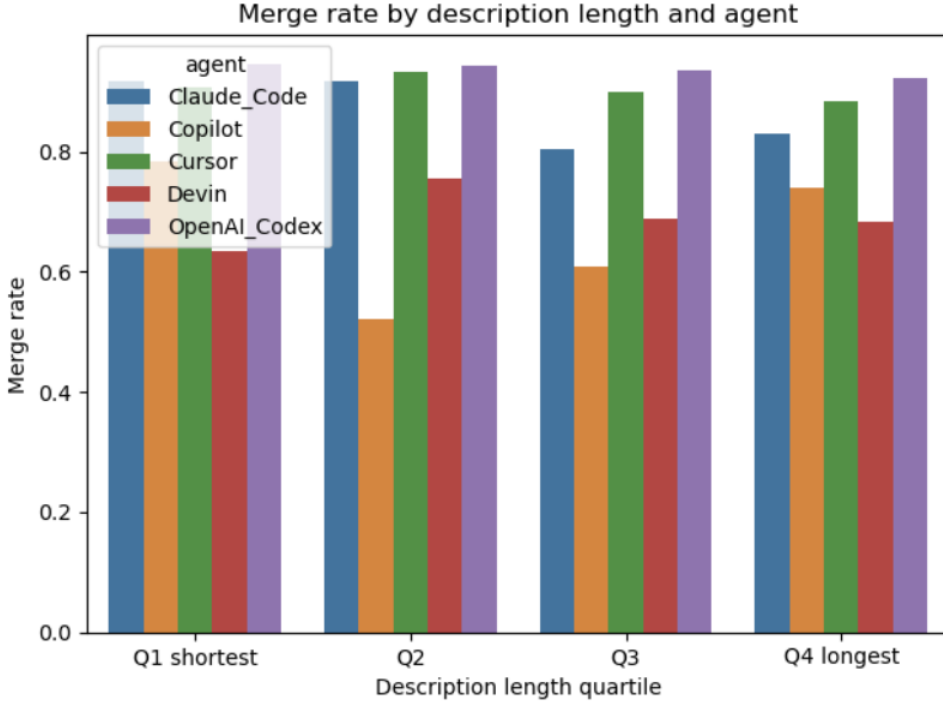


Fig. 2. Merge rate by description length and agent

was. Looking back, this part actually makes sense. If the code changes are already clear enough, reviewers probably don't need a long explanation anyway.

Another important point is that the agents had different numbers of PRs. Codex created many more PRs than some of the others, which led to smoother results from month to month. Agents with fewer PRs, especially in the early months, produced noisier patterns simply because small counts make each merge matter more. This means that not all changes in the data show real differences in behavior. Some are just random noise.

In summary, these observations suggest that performance depends on the agent's quality, how reviewers act, and how much each agent participates over time. Understanding all three factors helps explain why some trends are clear and others are less certain.

5 Limitations

This project focuses only on merged vs. closed PRs and does not consider code content or review comments. More detailed semantic analysis could reveal why certain PRs fail. Also, the dataset spans only eight months, so longer-term trends cannot be tested. Finally, heavy outliers in merge time might be due to repository-specific delays unrelated to the agents themselves.

6 Conclusion

Basically, the results show that different AI coding agents don't perform the same when it comes to PR reviews on GitHub.

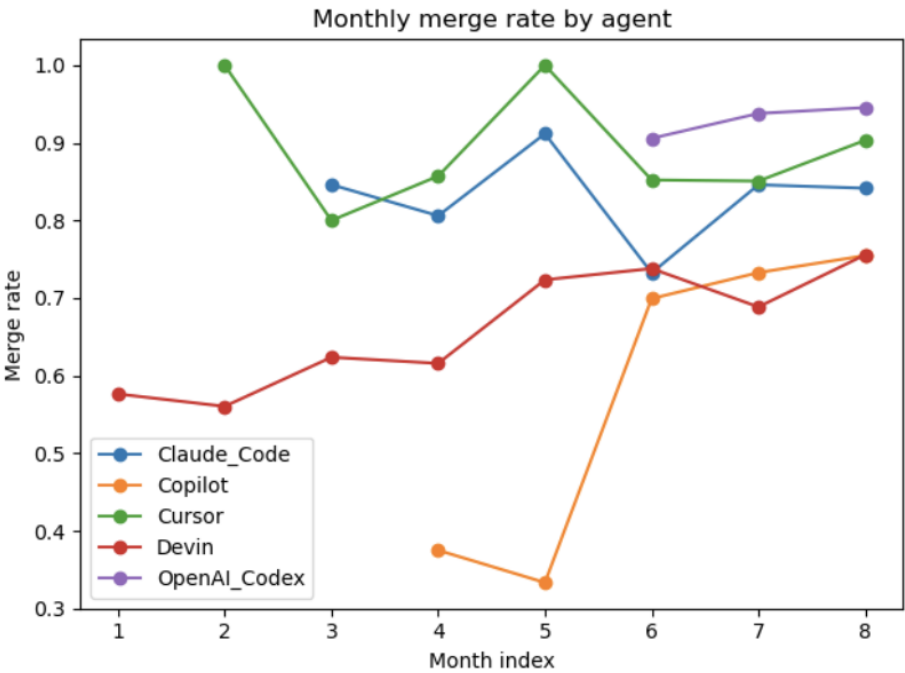


Fig. 3. Monthly merge rate by agent (Dec 2024–Jul 2025)

OpenAI_Codex performs best in terms of merge success and shows the clearest pattern of improvement. Description length helps to some degree, but mainly for specific agents such as Cursor and Copilot. Trend analysis also hints that several agents may be getting better over time. These findings may be useful for developers or team considering which AI tools fit their workflows. Future work could explore reviewer behavior, code quality, or repository-level differences.

7 Project Details

7.1 Team Roles

Tao Wang: I am the Sole author. I do everything: research question formulation, data cleaning, Python coding (using pandas,scipy,statsmodels), statistical analysis, visualization, and report writing.

7.2 GitHub Repository

- GitHub Repository: https://github.com/UBCTAO/DATA542_FinalProject
- Dataset: HuggingFace AIDev Dataset (<https://huggingface.co/datasets/hao-li/AIDev>)

7.3 Generative AI Usage Statement

This is my first time writing a project by using LaTeX, and I ran into several LaTeX errors during the writing process (for example, issues with inserting figures and formatting). I used GenAI tools such as ChatGPT 5.1 or Gemini 3 pro to help me understand and fix these LaTeX formatting errors.

References

[1] Hao Li, et al. 2024. *AIDev: A Dataset of AI-Generated Pull Requests*. HuggingFace. Available at: <https://huggingface.co/datasets/hao-li/AIDev>.